

Machine learning in finance : Theoretical foundations

Huyên PHAM

Ecole Polytechnique, CMAP

<https://sites.google.com/site/phamxuanhuyen/>

Email : huyen.pham@polytechnique.edu or pham@ensae.fr

Master M2MO, SFA
Mastère spécialisé FGR, DS
ENSAE 3A
Year 2025-2026

Lecture 2 : Neural networks-based algorithms for high-dimensional problems

- 1 PDEs and stochastic control in finance
- 2 Deep learning approaches for PDEs
- 3 Deep learning for stochastic control

Outline

- 1 PDEs and stochastic control in finance
- 2 Deep learning approaches for PDEs
- 3 Deep learning for stochastic control

PDEs are universal in modelling

- Partial Differential Equations (PDEs) :

$$\partial_t v(t, x) = \mathcal{H}[v](t, x) := H(t, x, v, D_x v, \dots), \quad (+ \text{ boundary conditions})$$

are among the most ubiquitous tools in modelling problems in nature (physics, biology, chemistry), and appear also in man-made complex systems, notably in financial engineering :

- Pricing and hedging of financial derivatives
- Portfolio optimization, algo trading
- Risk management

Diagram illustrating the Black-Scholes PDE and its components:

- $\frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2}$: Volatility, Rate of change, and rate of changes.
- $+ rS \frac{\partial V}{\partial S}$: Price of financial derivative.
- $+ \frac{\partial V}{\partial t}$: Rate of change.
- $- rV = 0$: Time, Risk-free interest rate.
- $\frac{\partial V}{\partial S}$: Delta (Sensitivity).
- $\frac{\partial^2 V}{\partial S^2}$: Gamma (Sensitivity).
- σ : Volatility.
- r : Risk-free interest rate.
- S : Stock price.
- V : Option price.
- t : Time.

Key Message

In finance, PDEs arise from **no-arbitrage** (pricing) or from **optimality principles** (control).

Linear PDEs in finance (no control/optimization)

- Linear PDEs¹

$$\begin{cases} \frac{\partial v}{\partial t} - rv + \underbrace{b(x) \cdot D_x v + \frac{1}{2} \sigma \sigma^\top(x) : D_x^2 v}_{\mathcal{L}[v]} + f(x) = 0, & \text{on } [0, T) \times \mathbb{R}^d, \\ v(T, x) = g(x), & x \in \mathbb{R}^d, \end{cases} \quad (1)$$

$\leftrightarrow$ Option pricing under Feynman-Kac representation :

$$v(t, x) = \mathbb{E}_{t,x} \left[\int_t^T e^{-r(s-t)} f(X_s) ds + e^{-r(T-t)} g(X_T) \right], \quad (2)$$

where X is the diffusion process (asset price, stochastic volatility, factor, etc) $\leftrightarrow$ linear differential operator $\mathcal{L}$:

$$dX_s = b(X_s)ds + \sigma(X_s)dW_s.$$

1. Inner product of two matrices $A : B = \text{tr}(A^\top B)$.

Variational inequalities

- Free boundary problem

$$\begin{cases} \min \left[-\frac{\partial v}{\partial t} + rv - \mathcal{L}[v] - f, v - g \right] = 0, & \text{on } [0, T) \times \mathbb{R}^d, \\ v(T, x) = g(x), & x \in \mathbb{R}^d, \end{cases}$$

$\Leftrightarrow$ Optimal stopping time problem (e.g. [American option pricing](#)) :

$$v(t, x) = \sup_{\tau \in \mathcal{T}_{t, T}} \mathbb{E}_{t, x} \left[\int_t^T e^{-r(s-t)} f(X_s) ds + e^{-r(\tau-t)} g(X_\tau) \right],$$

where X is the diffusion process $\Leftrightarrow$ linear differential operator $\mathcal{L}$.

$\rightarrow$ Continuation region vs Exercise region.

Quasilinear PDEs

- Quasilinear PDE

$$\begin{cases} \frac{\partial v}{\partial t} + \mathcal{L}[v] + f(x, v) = 0, & \text{on } [0, T) \times \mathbb{R}^d, \\ v(T, x) = g(x), & x \in \mathbb{R}^d. \end{cases}$$

Example : CVA (Credit Valuation Adjustment) pricing

$$f(x, y) = r \max(y, 0) - ry.$$

$r > 0$ coincides with the intensity of default of a counterparty.

See details in [Burgard, Kjaer](#), Journal of Credit Risk, 2011

Semilinear PDEs

- Semilinear PDE

$$\begin{cases} \frac{\partial v}{\partial t} + \mathcal{L}[v] + f(x, v, D_x v) = 0, & \text{on } [0, T) \times \mathbb{R}^d, \\ v(T, x) = g(x), & x \in \mathbb{R}^d. \end{cases}$$

Applications :

- Initial Margin Valuation Adjustment (MVA) : see [Green, Kenyon](#), arXiv: 1405.0508
- Bellman equation, e.g.,

$$f(x, y, z) = -ry + \sup_{a \in A} [a \cdot z + h(x, a)].$$

$\leftrightarrow$ stochastic control problem with controlled drift (e.g. portfolio liquidation)

$$\begin{aligned} v(t, x) &= \sup_{\alpha} \mathbb{E}_{t,x} \left[\int_t^T e^{-r(s-t)} h(X_s, \alpha_s) ds + e^{-r(T-t)} g(X_T) \right] \\ dX_s &= [b(X_s) + \alpha_s] ds + \sigma(X_s) dW_s. \end{aligned}$$

Fully nonlinear PDEs

- Nonlinear PDE

$$\begin{cases} \frac{\partial v}{\partial t} + F(x, v, D_x v, D_x^2 v) = 0, & \text{on } [0, T) \times \mathbb{R}^d, \\ v(T, x) = g(x), & x \in \mathbb{R}^d. \end{cases}$$

Applications :

- Bellman equation

$$F(x, y, z, \gamma) = -ry + \sup_{a \in A} [b(x, a) \cdot z + \frac{1}{2} \sigma \sigma^\top(x, a) : \gamma + h(x, a)].$$

$\leftrightarrow$ stochastic control problem with both controlled drift and volatility (e.g. portfolio optimization)

$$\begin{aligned} v(t, x) &= \sup_{\alpha} \mathbb{E}_{t,x} \left[\int_t^T e^{-r(s-t)} h(X_s, \alpha_s) ds + e^{-r(T-t)} g(X_T) \right] \\ dX_s &= b(X_s, \alpha_s) ds + \sigma(X_s, \alpha_s) dW_s. \end{aligned}$$

Hierarchy of PDEs in Finance

- Linear PDE $\rightarrow$ Pricing (Feynman–Kac)
- Variational inequality $\rightarrow$ Optimal stopping (American options)
- Semilinear PDE $\rightarrow$ Control with drift control (optimal execution)
- Fully nonlinear PDE $\rightarrow$ Control with drift and volatility control (portfolio selection)

Increasing complexity

Linearity $\rightarrow$ Nonlinearity $\rightarrow$ Optimization $\rightarrow$ Curse of dimensionality

Stochastic control

- Stochastic control deals with the control of dynamical systems in a random environment. Many applications in finance, e.g. portfolio allocation, optimal execution, algorithmic trading, etc

Continuous vs Discrete Time

Continuous time $\rightarrow$ HJB PDE

Discrete time $\rightarrow$ Markov Decision Process (MDP)

Same optimality principle, different mathematical formulation.

Example : Trade execution problem

- Want to sell N shares of a given stock within a time period (say 2mn, with trading every 30 sec) so as to maximize revenue in a limit order book

► State variables :

- Endogenous (controlled) variables : remaining inventory
- Exogenous (market) variables :
 - bid-ask spread
 - transaction volume
 - ...

► Action

- Limit order price
- Number of shares

► Reward : cash inflow following execution of an order

Numerical challenges

- **Grid based methods** : mesh size (and cost) scales exponentially with dimension (*curse of dimensionality*)
- **Monte-Carlo/regression methods** :
 - Efficiency limited to $d = 5, 6, \dots, 10$
 - Solution computed (essentially) at a single spatial point, but not on the whole domain

Motivation

Neural networks approximate functions in high dimension without discretizing the state space.

Outline

- 1 PDEs and stochastic control in finance
- 2 Deep learning approaches for PDEs
- 3 Deep learning for stochastic control

Deep Learning Approach to PDEs : Basic Principle

We consider a PDE for v on $[0, T] \times \mathbb{R}^d$.

Step 1 — Function approximation

- Approximate the unknown solution $v(t, x)$ by a neural network :

$$v(t, x) \approx U_\theta(t, x)$$

- Automatic differentiation provides $D_x U_\theta$, $D_x^2 U_\theta$, $\partial_t U_\theta$

Step 2 — Optimization

- Define a loss function measuring PDE residual + terminal condition
- Minimize over parameters θ using stochastic gradient descent
- Various numerical schemes depending on loss function/training simulation/deterministic vs probabilistic approach

Paradigm shift :

- Classical solvers discretize the *state space*
- Deep learning discretizes the *function space*

Reformulating the PDE as an Optimization Problem

Consider the PDE :

$$\begin{cases} \partial_t v(t, x) = \mathcal{H}[v](t, x), & (t, x) \in [0, T) \times \mathbb{R}^d \\ v(T, x) = g(x) \end{cases}$$

Idea : A function v solves the PDE if and only if it minimizes the residual error.

Define the loss functional :

$$\mathbb{L}(w) = \mathbb{E}[|w(T, X) - g(X)|^2] + \mathbb{E}[|\partial_t w(\tau, X) - \mathcal{H}[w](\tau, X)|^2]$$

where $(\tau, X) \sim \nu_T \otimes \nu_d$ training measures on $[0, T] \times \mathbb{R}^d$.

$$v \text{ solves the PDE} \iff \mathbb{L}(v) = \inf_w \mathbb{L}(w) = 0. \quad (3)$$

Problem (3) is still an **infinite dimensional** problem, hence not yet amenable to numerical computations.

Neural Network Approximation

The problem remains infinite-dimensional.

Approximation step :

$$v(t, x) \approx U_\theta(t, x)$$

where U_θ is a neural network.

→ This reduces the problem to finite-dimensional optimization :

$$\inf_{\theta} L(\theta), \quad L(\theta) := \mathbb{L}(U_\theta)$$

Automatic differentiation allows computation of :

$$\partial_t U_\theta, \quad D_x U_\theta, \quad D_x^2 U_\theta$$

Stochastic Optimization : DGM (Deep Galerkin) and PINN (Physics-Informed Neural networks)

The loss can be written as an expectation :

$$L(\theta) = \mathbb{E}[\ell(\theta, \tau, X)]$$

with

$$\ell(\theta, t, x) = |U_\theta(T, x) - g(x)|^2 + |\partial_t U_\theta(t, x) - \mathcal{H}[U_\theta](t, x)|^2.$$

We minimize using stochastic gradient descent :

$$\theta^{(k+1)} = \theta^{(k)} - \eta_k \frac{1}{N} \sum_{n=1}^N \nabla_{\theta} \ell(\theta^{(k)}, \tau_{k,n}, X_{k,n}).$$

Difference between DGM and PINN :

- DGM (Sirignano, Spiliopoulos 2017) : i.i.d. random sampling in the domain
- PINN (Raissi et al. 19) : collocation points (possibly fixed or adaptive)

Pros and Cons : DGM vs PINN

DGM (Monte Carlo residual minimization)

Pros :

- Scales naturally to high dimension
- Flexible stochastic sampling

Cons :

- Residual minimization does not control pointwise error or gradient accuracy.
- Sensitive to sampling distribution
- Optimization instability

PINN (collocation-based)

Pros :

- Strong enforcement at selected points
- Effective in low dimension
- Widely used in physics applications

Cons :

- Curse of dimensionality
- Loss imbalance and approximation frequency problem

From Deterministic to Probabilistic Methods

Two perspectives on nonlinear PDEs :

Deterministic :

- Minimize residual in the domain
- DGM / PINN

Probabilistic :

- Represent PDE via BSDE
- Learn solution along simulated trajectories

This leads to Deep BSDE and Deep BDP methods.

Reminder : option pricing and PDE (Feynman-Kac formula)

Option pricing theory : perfect replication of a payoff $g(X_T)$ consists in finding (Y, Z) solution to $(r = 0)$

$$g(X_T) = Y_0 + \int_0^T Z_t dX_t$$

with

- $Y_t = v(t, X_t) = \mathbb{E}[g(X_T) | \mathcal{F}_t]$: arbitrage price at time t
- $Z_t = D_x v(t, X_t)$: Delta hedging at time t

and v solves the linear PDE

$$\begin{cases} \frac{\partial v}{\partial t} + \mathcal{L}v = 0, & \text{on } [0, T) \times \mathbb{R}^d, \\ v(T, x) = g(x), & x \in \mathbb{R}^d, \end{cases}$$

where $\mathcal{L}$ is the generator of the martingale price process X (under risk neutral measure).

Hedging with NN : a supervised learning approach

- **Replication revisited** : find Y_0 and Neural networks $\mathcal{Z}^\theta(t, x)$, that minimize the loss function (hedging error) :

$$J(Y_0, \theta) = \mathbb{E} \left| g(X_T) - Y_0 - \int_0^T \mathcal{Z}^\theta(t, X_t) dX_t \right|^2$$

by training simulations of input price X .

→ Output :

- $\hat{Y}_0$: approximation of option price $v(0, X_0)$ at time 0
- $\hat{\mathcal{Z}}(t, x) = \mathcal{Z}^{\hat{\theta}}(t, x)$: approximation of Delta $D_x v(t, x)$
- **NN as control variate** : $\hat{\mathcal{Z}}$ can be viewed as control variates for the Monte-Carlo estimator of $\mathbb{E}[g(X_T)]$ as

$$\begin{aligned} \mathbb{E} \left[g(X_T) - \int_0^T \hat{\mathcal{Z}}(t, X_t) dX_t \right] &= \mathbb{E}[g(X_T)] \\ \text{Var} \left[g(X_T) - \int_0^T \hat{\mathcal{Z}}(t, X_t) dX_t \right] &\simeq \text{Var}[Y_0] = 0, \end{aligned}$$

Example 1 : low dimensional problem with explicit solution

Exchange option on two assets in a Black-Scholes model :

$$g(S) = \max(S^1 - S^2, 0),$$

→ Explicit option price given by Margrabe formula.

Method	Emp. Var.	Var. Red. Fact.	Iter.
Monte-Carlo	3.16×10^{-2}		
MC + CV NN	2.59×10^{-4}	121.98	1389
MC + CV Margrabe	2.12×10^{-4}	149.19	

Table – Results on exchange option on two assets. Threshold criteria for stopping iterations : $\varepsilon = 5 \times 10^{-6}$.

Example 2 : high dimensional problem, exchange against average

$d = 100$, and option payoff : $g(S) = \max(S^1 - \frac{1}{d-1} \sum_{i=2}^d S^i, 0)$.

Method	Emp. Var.	Var. Red. Fact.	Iter.
Monte-Carlo	1.97×10^{-2}		
MC + CV NN	1.51×10^{-4}	130.39	2829

Table – Results on exchange option with 100 assets. Threshold criteria for stopping iterations : $\varepsilon = 5 \times 10^{-6}$.

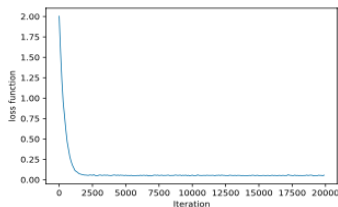


Figure – Value of loss function in terms of number of iterations

Pushing further the replication approach : PDE/BSDE

Semi-linear PDE (e.g. in CVA pricing, market frictions, optimal execution problem) :

$$\begin{cases} \partial_t v + \mathcal{L}[v] = f(x, v, D_x v), & \text{on } [0, T) \times \mathbb{R}^d, \\ v(T, x) = g(x), & x \in \mathbb{R}^d, \end{cases} \quad (4)$$

Key Idea

Instead of minimizing residual in the whole domain, learn the solution along simulated trajectories.

Forward simulation + backward learning.

BSDE representation

- Introduce the forward process $X \leftrightarrow \mathcal{L}$, and consider the pair process (Y, Z) :

$$Y_t = v(t, X_t), \quad Z_t = D_x v(t, X_t), \quad 0 \leq t \leq T.$$

→ By Itô's formula :

$$\underbrace{v(T, X_T)}_{g(X_T)} = \underbrace{v(t, X_t)}_{Y_t} + \int_t^T \underbrace{\left(\frac{\partial v}{\partial t} + \mathcal{L}[v] \right)(s, X_s)}_{f(X_s, Y_s, Z_s)} ds + \int_t^T \underbrace{D_x v(s, X_s)^\top \sigma(X_s)}_{Z_s^\top \sigma(X_s)} dW_s$$

→ **Probabilistic representation** of (4) :

$$g(X_T) = Y_t + \int_t^T \underbrace{f(X_s, Y_s, Z_s)}_{\text{red}} ds + \int_t^T Z_s^\top \sigma(X_s) dW_s, \quad 0 \leq t \leq T \quad (5)$$

i.e. in differential form

$$dY_t = f(X_t, Y_t, Z_t)dt + Z_t^\top \sigma(X_t)dW_t, \quad Y_T = g(X_T), \quad (6)$$

called **backward stochastic differential equation** (BSDE) : Pardoux, Peng (90), El Karoui, Peng, Quenez (97) : extension of Feynman-Kac formula

Reformulating PDE as a stochastic optimization problem

- Given $y_0 \in \mathbb{R}$, and $\mathcal{Z} = (Z_t)_t$ square-integrable adapted process, let us define the forward process from the BSDE dynamics :

$$Y_t^{y_0, \mathcal{Z}} = y_0 + \int_0^t f(X_s, Y_s^{y_0, \mathcal{Z}}, \mathcal{Z}_s) ds + \int_0^t \mathcal{Z}_s^\top \sigma(X_s) dW_s, \quad 0 \leq t \leq T.$$

and the loss function :

$$\mathbb{L}(y_0, \mathcal{Z}) = \mathbb{E} |Y_T^{y_0, \mathcal{Z}} - g(X_T)|^2,$$

- By taking the pair (Y_0, Z) , with $Y_0 = v(0, X_0)$, $Z_t = D_x v(t, X_t)$, with v solution to the PDE, we see that $(Y^{Y_0, Z}, Z)$ and (Y, Z) solves the BSDE dynamics (6), starting from same initial value Y_0 , hence by uniqueness : $Y^{Y_0, Z} = Y$, and so $Y_T^{Y_0, Z} = g(X_T)$:
 $\rightarrow (Y_0, Z)$ is solution to the stochastic optimization (SA) problem :

$$\mathbb{L}(Y_0, Z) = 0 = \inf_{y_0, \mathcal{Z}} \mathbb{L}(y_0, \mathcal{Z}).$$

Deep BSDE method (Han, E. Jentzen 17)

- Idea : approximate the unknown functions $X_0 \mapsto Y_0 = v(0, X_0)$ and $(t, X_t) \mapsto Z_t = D_x v(t, X_t)$ by neural networks $\mathcal{U}_\theta$ on $\mathbb{R}^d$ and $\mathcal{Z}_\theta$ on $[0, T] \times \mathbb{R}^d$.

► A supervised learning approach : simulate the BSDE dynamics **forward in time** starting from $\mathcal{U}_\theta(X_0)$ and using $\mathcal{Z}_\theta$:

$$Y_t^\theta = \mathcal{U}_\theta(X_0) + \int_0^t f(X_s, Y_s^\theta, \mathcal{Z}_\theta(s, X_s)) ds + \int_0^t \mathcal{Z}_\theta(s, X_s)^\top \sigma(X_s) dW_s, \quad 0 \leq t \leq T,$$

and minimize **terminal error** :

$$L(\theta) = \mathbb{E} |Y_T^\theta - g(X_T)|^2$$

Remarks.

- In practice, discretize using Euler scheme on a grid $t_0 = 0 < t_1 < \dots < t_n = T$ with time step Δt : starting from $Y_0^\theta = \mathcal{U}_\theta(X_0)$,

$$Y_{t_{i+1}}^\theta = Y_{t_i}^\theta + f(X_{t_i}, Y_{t_i}^\theta, \mathcal{Z}_\theta(t_i, X_{t_i})) \Delta t + \mathcal{Z}_\theta(t_i, X_{t_i})^\top \sigma(X_{t_i}) \Delta W_{t_i}, \quad i = 0, \dots, n-1.$$

- The networks take the paths $(X_{t_i})_{0 \leq i \leq n}$ and $(W_{t_i})_{0 \leq i \leq n}$ (easily simulated) as input data, are trained via SGD, and yield as output $\mathcal{U}_{\theta^*}, \mathcal{Z}_{\theta^*}$ approximations of $v(0, \cdot)$ and $D_x v$, and also $Y_{t_i}^{\theta^*}$ approximation of $v(t_i, X_{t_i})$.

Numerical implementation

- Parametrized Neural Networks : $\mathcal{U}_0 = \mathcal{U}_0^\theta(\cdot)$, $\mathcal{Z} = \mathcal{Z}^\theta(\cdot)$
- Expectation in loss function replaced by empirical average : $\rightarrow$ Stochastic Gradient Descent (SGD) with mini-batch size K

$$J_G^K(\theta) = \frac{1}{K} \sum_{k=1}^K \left| g(X_T(\omega^k)) - Y_N^{\mathcal{U}_0^\theta, \mathcal{Z}^\theta}(\omega^k) \right|^2$$

where $X(\omega^k)$ is the k -th sample of the forward process X (possibly via Euler scheme) : training data set.

Comments on Deep BSDE

- A forward approach.
- + Simple to implement
- + Idea of approximating strategies at any time by NN, and optimizing a global criterion by model simulations $\rightarrow$ Deep hedging (Bühler, Gonon, Teichmann, Wood 19)
- Approximation of the solution only along trajectories (X_{t_i})
- In the time discretization, there is a subnetwork at each time $t_i \rightarrow$ huge neural network NN composed of Deep NN at each time step with parameters $\theta = (\theta_0, \dots, \theta_{N-1}) \rightarrow$ unstable and may diverge :
 - Global learning of the optimal parameters can be a daunting task with large time steps n
 - SGD over the global DNN may be trapped in local minimizer
- It is better to stack all the subnetworks to form a DNN as a whole, i.e. a DNN on $[0, T] \times \mathbb{R}^d$.

From BSDE to Dynamic Programming

The BSDE discretization induces a backward recursion : starting from $Y_T = g(X_T)$,

$$\begin{aligned} Y_{t_i} &= Y_{t_{i+1}} - f(X_{t_i}, Y_{t_i}, Z_{t_i})\Delta t_i - Z_{t_i}^\top \sigma(X_{t_i})\Delta W_{t_i} \\ &= \mathbb{E}\left[Y_{t_{i+1}} - f(X_{t_i}, Y_{t_i}, Z_{t_i})\Delta t_i \middle| \mathcal{F}_{t_i}\right], \quad i = N-1, \dots, 0. \end{aligned} \quad (7)$$

Observation

BSDE discretization naturally induces a backward dynamic programming structure.

Can we directly approximate the value function and its gradient backward in time?
(recall $Y_{t_i} = v(t_i, X_{t_i})$, $Z_{t_i} = \nabla_x v(t_i, X_{t_i})$)

Deep Backward Dynamic Programming (Deep BDP) : (Huré, Pham, Warin 2020)

Instead of learning (Y_0, Z_{t_i}) globally :

- Approximate value function and gradient at each time step t_i :

$$v(t_i, x) \approx \mathcal{U}_{\theta_i}(x), \quad \nabla_x v(t_i, x) \approx \mathcal{Z}_{\theta_i}(x),$$

- Use backward induction (7) : starting from $v(t_N, x) = g(x)$,

$$\begin{aligned} v(t_i, X_{t_i}) = & v(t_{i+1}, X_{t_{i+1}}) - f(X_{t_i}, v(t_i, X_{t_i}), \nabla_x v(t_{i+1}, X_{t_{i+1}}))\Delta t_i \\ & - \nabla_x v(t_{i+1}, X_{t_{i+1}})^\top \sigma(X_{t_i})\Delta W_{t_i} \end{aligned}$$

Dynamic programming + neural approximation.

Algo DBDP1 : Representation of Y and Z by DNN

Algorithm Deep BDP1 Algorithm

- 1: Initialize $\hat{\mathcal{U}}_N^{(1)} = g$
- 2: For $i = N - 1, \dots, 0$: minimize over NN $(\mathcal{U}, \mathcal{Z})$ the loss function

$$\mathbb{E} \left| \hat{\mathcal{U}}_{i+1}^{(1)}(X_{t_{i+1}}) - \mathcal{U}(X_{t_i}) - f(X_{t_i}, \mathcal{U}(X_{t_i}), \mathcal{Z}(X_{t_i})) \Delta t_i - \mathcal{Z}(X_{t_i})^\top \sigma(X_{t_i}) \Delta W_{t_i} \right|^2$$

by training simulations of (X_{t_i}) .

Update $(\hat{\mathcal{U}}_i^{(1)}, \hat{\mathcal{Z}}_i^{(1)})$ as the solution to this minimization problem

Output :

- $(\hat{\mathcal{U}}_i^{(1)})_{i=0}^N$: Estimates of $v(t_i, \cdot)$ for $i = 0, \dots, N$
- $(\hat{\mathcal{Z}}_i^{(1)})_{i=0}^N$: Estimates of $D_x v(t_i, \cdot)$ for $i = 0, \dots, N$;

Algo DBDP2 : Representation of Y only by DNN

Algorithm Deep BDP2 Algorithm

- 1: Initialize $\hat{\mathcal{U}}_N^{(2)} = g$
- 2: For $i = N - 1, \dots, 0$: minimize over C^1 NN $\mathcal{U}$ the loss function

$$\mathbb{E} \left| \hat{\mathcal{U}}_{i+1}^{(1)}(X_{t_{i+1}}) - \mathcal{U}(X_{t_i}) - f(X_{t_i}, \mathcal{U}(X_{t_i}), D_x \mathcal{U}(X_{t_i})) \Delta t_i - D_x \mathcal{U}(X_{t_i})^\top \sigma(X_{t_i}) \Delta W_{t_i} \right|^2$$

by training simulations of (X_{t_i}) .

Update $\hat{\mathcal{U}}_i^{(2)}$ as the solution to this minimization problem

► Output :

- $(\hat{\mathcal{U}}_i^{(2)})_{i=0}^N$: Estimates of $v(t_i, \cdot)$ for $i = 0, \dots, N$;
- $(D_x \hat{\mathcal{U}}_i^{(2)})_{i=0}^N$: Estimates of $D_x v(t_i, \cdot)$ for $i = 0, \dots, N$.

Extension to variational inequalities : Deep RBDP

Algorithm Deep Reflected BDP

- 1: Initialize $\hat{\mathcal{U}}_N^{(R)} = g$
- 2: For $i = N - 1, \dots, 0$: minimize over NN $(\mathcal{U}, \mathcal{Z})$ the loss function

$$\mathbb{E} \left| \hat{\mathcal{U}}_{i+1}^{(R)}(X_{t_{i+1}}) - \mathcal{U}(X_{t_i}) - f(X_{t_i}, \mathcal{U}(X_{t_i}), \mathcal{Z}(X_{t_i}))\Delta t_i - \mathcal{Z}(X_{t_i})^\top \sigma(X_{t_i})\Delta W_{t_i} \right|^2$$

by training simulations of (X_{t_i}) .

Set $(\hat{\mathcal{U}}_i, \hat{\mathcal{Z}}_i)$ as the solution to this minimization problem and update

$$\hat{\mathcal{U}}_i^{(R)} = \max[\hat{\mathcal{U}}_i, g]$$

Output : $(\hat{\mathcal{U}}_i^{(R)})_{i=0}^N$: Estimates of $v(t_i, \cdot)$ for $i = 0, \dots, N$

Comments on Deep BDP

- A backward approach.
- N local (at each time step t_i) minimization problems vs a global minimization in deep BSDE and deep Galerkin. May be costly but in practice :
 - + **Transfer learning** : at each time t_i , initialize the parameters of the NN to the "optimal" parameters of the time t_{i+1} previously treated. This **reduces significantly** the number of iterations in SGD after the first step at time t_{N-1}
 - + less prone to be trapped in local minimizer, hence more stable (in particular for T large) compared to deep BSDE

Deep BSDE vs Deep BDP

Method	Deep BSDE	Deep BDP
Learning	Global (Y_0, Z_t)	Backward value functions
Structure	Forward simulation	Backward recursion
Stability	Good	Often more stable
Interpretation	Stochastic representation	Dynamic programming
Approximation domain	Along trajectories	Whole state space

Two complementary probabilistic approaches.

Test 1 : a semi-linear PDE with quadratic gradient term

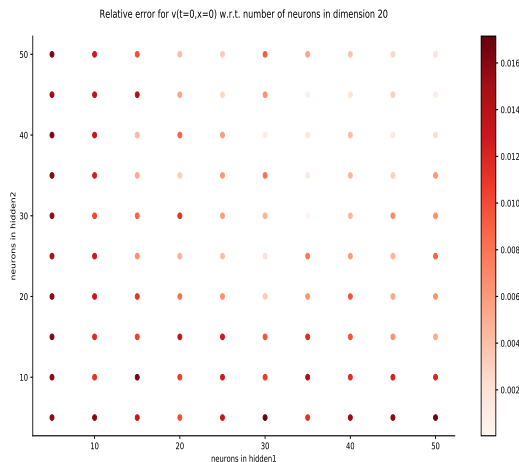
- $f(x, y, z) = \frac{1}{2}|z|^2$:

$$\begin{cases} \frac{\partial v}{\partial t} + \frac{1}{2}\Delta_x v &= \frac{1}{2}|D_x v|^2, \quad (t, x) \in [0, T) \times \mathbb{R}^d \\ v(T, x) &= g(x) \end{cases}$$

→ Explicit solution (via Hopf-Cole transformation) :

$$v(0, x_0) = -\ln \left(\mathbb{E} \left[\exp \left(-g(x_0 + W_T) \right) \right] \right).$$

Numerical results : $d = 20$, $L = 2$ hidden layers, and varying the number K of neurons



Further numerical tests

- $L = 2$ hidden layers with $K = d + 10$ neurons
- $\sigma = \tanh$ as activation function
- Adam optimizer in TensorFlow and mini-batch with 1000 trajectories for SGD

Test 2a : An example of PDE with simple structure, $T = 1$, $N = 120$

$$\begin{cases} f(t, x, y, z) &= -\left(\cos(\bar{x})\left(e^{\frac{T-t}{2}} + \frac{1}{2}\right) + 0.2 \sin(\bar{x})\right)e^{\frac{T-t}{2}} \\ &\quad + \frac{1}{2}\left(\sin(\bar{x})\cos(\bar{x})e^{T-t}\right)^2 - \frac{1}{2d^2}(yz \cdot 1_d)^2 \\ g(x) &= \cos(\bar{x}) \end{cases}$$

with $\bar{x} = \sum_{k=1}^d x_k$. $\rightarrow$ explicit analytic solution $v(t, x) = e^{\frac{T-t}{2}} \cos(\bar{x})$.

► Mean and standard deviation over 10 independent runs.

	Dimension d	Mean value	Standard deviation
Exact solution	5	0.46768	
Deep BDP1	5	0.4637038	0.004253
Deep BDP2	5	0.46335	0.00137
Deep BSDE	5	0.46562	0.0035
Exact solution	20	0.6728	
Deep BDP1	20	0.6760	0.00274
Deep BDP2	20	0.67102	0.00559
Deep BSDE	20	0.68686	0.002402
Exact solution	50	1.5909	
Deep BDP1	50	1.5903	0.006276
Deep BDP2	50	1.58762	0.00679
Deep BSDE	50	1.583023	0.00361

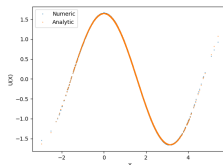
Table – Estimate of $v(0, x_0)$ with $x_0 = 1_d$.

Test 2b : An example of PDE with simple structure, $T = 2$, $N = 240$

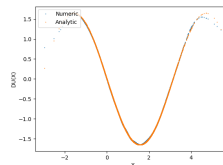
- $d = 1$, Estimate of $v(0, x_0)$ with $x_0 = 1$. Mean and Std observed over 10 independent runs.

	Averaged value	Standard deviation
Exact solution	1.4686938	
Deep BDP1	1.46332	0.01434
Deep BDP2	1.4387982	0.01354
Deep BSDE	NC	NC

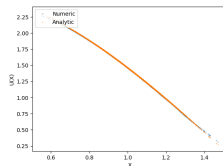
Test 2b : Plots of $x \mapsto v(t, \cdot)$ and $D_x v(t, x)$ by Deep BDP1



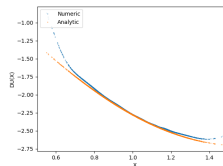
$$x \mapsto v(t = 1, x)$$



$$x \mapsto D_x v(t = 1, x)$$



$$x \mapsto v(t = 0.0091, x)$$



$$x \mapsto D_x v(t = 0.0091, x)$$

Numerical tests for CVA pricing

CVA in a d -dimensional Black-Scholes model of a straddle payoff :

$$\begin{cases} \partial_t u + \frac{\sigma^2}{2} \text{tr}(x^\top D_x^2 u x) + \beta(u_+ - u) = 0 & \text{on } [0, T] \times \mathbb{R}^d \\ u(T, x) = |\sum_{i=1}^d x_i - d| - 0.1 & \text{on } \mathbb{R}^d \end{cases} \quad (8)$$

Dimension d	Deep BDP	Deep BSDE
1	0.05950 (0.000257)	0.05949 (0.000264)
3	0.17797 (0.000421)	0.17807 (0.000288)
5	0.25956 (0.000467)	0.25984 (0.000331)
10	0.40930 (0.000623)	0.40886 (0.000196)
15	0.52353 (0.000591)	0.52389 (0.000551)
30	0.78239 (0.000832)	0.78231 (0.001266)

Table – CVA value with $X_0 = 1$, $T = 1$, $\beta = 0.03$, $\sigma = 0.2$ and 50 time steps. Results are averaged over 10 runs and Std in parentheses.

Numerical tests for American option pricing

d assets of price $(X^1, \dots, X^d) \rightsquigarrow$ Multi-dimensional BS model

- Parameters : $r = 0.05$, $\sigma_i = 0.2$, $K = 1$, $X_0^i = 1$, $i = 1, \dots, d$, Geometric Put option : $g(X_T) = (K - \prod_{i=1}^d X_T^i)_+$, $T = 1$
- Numerical results for deep RBDP :

Dimension	nb steps N	value	std	bench
1	40	0.061122	0.00015	0.060903
5	40	0.10725	0.00012	0.10738
10	80	0.129923	0.00016	0.12996
20	80	0.15050	0.00010	0.1510
40	160	0.16758	0.00016	0.1680

Table – Estimate of the American option price using Deep RBDP. Results from 40 independent runs of simulations.

Outline

- 1 PDEs and stochastic control in finance
- 2 Deep learning approaches for PDEs
- 3 Deep learning for stochastic control**

Markov decision process (MDP)

MDP is defined by a quadruple $(\mathcal{X}, A, P, r = (f, g))$

- $\mathcal{X}$: **state** space in which the discrete-time process $(X_t)_{t \in \mathbb{N}}$ is valued
- A : **action** space in which the control/decision $(\alpha_t)_t$ is valued
- State dynamics determined by

$$X_{t+1} \sim P_t(X_t, \alpha_t)$$

with a **transition probability** $(t, x, a) \in \mathbb{N} \times \mathcal{X} \times A \mapsto P_t(x, a, dx') \in \mathcal{P}(\mathcal{X})$.

Example : When the state dynamics is driven by

$$X_{t+1} = F_t(X_t, \alpha_t, \varepsilon_{t+1}),$$

with i.i.d. noise $(\varepsilon_t)_t$, arising e.g. in time discretization of a controlled diffusion process, $\leftrightarrow P_t(x, a) \equiv \text{law of } F_t(x, a, \varepsilon_1)$.

- **Reward** (reinforcement) :
 - running reward $f(x, a)$ obtained in state x when choosing action a
 - terminal reward (case of finite horizon) $g(x)$
 - Discount factor $\beta \in [0, 1]$ (case of infinite horizon)

Policy

- A (Markovian) policy π is a sequence $(\pi_t)_t$ which can be either
 - **deterministic** when $\pi_t : \mathcal{X} \rightarrow A$, i.e., $\pi_t(x)$ represents the action value chosen at time t in state x
 - **randomized/stochastic** when $\pi_t : \mathcal{X} \rightarrow \mathcal{P}(A)$, i.e., $\pi_t(x, da)$ represents the probability of choosing an action at time t in state x
- We say that a control α is drawn (or follows) a policy π , denoted $\alpha \sim \pi$, when
 - $\alpha_t = \pi_t(X_t)$ (case of deterministic policy), for all t
 - $\alpha_t \sim \pi_t(X_t)$ (case of randomized policy), for all t
- Goal : learn a policy/control that maximizes the sum of rewards

Evaluation of a policy : value functions

- **State value function** (over a finite horizon $T \in \mathbb{N}^*$) of a policy $\pi = (\pi_t)_t$:

$$V_t^\pi(x) = \mathbb{E}_\pi \left[\sum_{s=t}^{T-1} f(X_s, \alpha_s) + g(X_T) | X_t = x \right], \quad t \in \{0, \dots, T\}, x \in \mathcal{X},$$

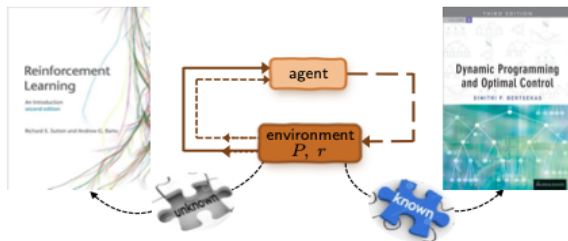
Here, $\mathbb{E}_\pi$ is expectation w.r.t. trajectory X when $\alpha \sim \pi$.

- **Q-function** of π :

$$Q_t^\pi(x, a) = \mathbb{E}_\pi \left[\sum_{s=t}^{T-1} f(X_s, \alpha_s) + g(X_T) | X_t = x, \alpha_t = a \right], \quad t \in \{0, \dots, T\}, (x, a) \in \mathcal{X} \times A.$$

Remark : $V_t^\pi(x) = \mathbb{E}_{a \sim \pi_t(x)} [Q_t^\pi(x, a)]$.

Searching for optimal policy



Goal : find an **optimal policy** π^* that maximize V^π

→ Optimal value function/ Q function :

$$V := \sup_{\pi} V^{\pi}, \quad Q := \sup_{\pi} Q^{\pi}$$

Remark : $V_t(x) = \sup_{a \in A} Q_t(x, a)$.

A key tool for MDP and RL : Dynamic programming (DP)

- MDP is a global dynamic optimization
- Dynamic programming is a method to reduce the optimization problem to local/static optimization problem

Bellman's principle of optimality (57) :

"An optimal policy has the property that, whatever the initial state and the initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision".

Bellman Principle of Optimality

Optimal value function :

$$\begin{cases} V_T(x) = g(x) \\ V_t(x) = \sup_{a \in A} \underbrace{\left\{ f(x, a) + \mathbb{E}_{x' \sim P_t(x, a)} [V_{t+1}(x')] \right\}}_{= Q_t(x, a)} \end{cases}$$

→ Optimal (deterministic) policy : $\pi^* = (\pi_t^*)$ from the **Q-value function** :

$$\pi_t^*(x) \in \arg \max_{a \in A} Q_t(x, a).$$

Key Idea

Global optimization reduces to a sequence of local problems.

Proof uses :

- Tower property of conditional expectation
- Backward induction

Full proof in appendix.

Annex

Computational Challenges

Two main difficulties :

- Computing conditional expectations

$$\mathbb{E}_{x' \sim P_t(x,a)}[V_{t+1}(x')]$$

in high dimension

- Optimizing over action space A

Curse of dimensionality in :

State space $\mathbb{R}^d$ and action space A .

Classical Probabilistic Approach

Backward Dynamic Programming :

For $t = T - 1, \dots, 0$:

- Approximate

$$Q_t(x, a) = f(x, a) + \mathbb{E}_{x' \sim P_t(x, a)}[V_{t+1}(x')]$$

via Monte-Carlo regression

- Compute

$$\pi_t(x) \in \arg \max_a Q_t(x, a)$$

- Update

$$V_t(x) = Q_t(x, \pi_t(x)).$$

Deep learning approach presented in this lecture

- Combine different ideas from maths (numerical probability) and machine learning communities to propose :

Two families of algorithms

1. Global policy optimization
2. Dynamic programming with local updates

Replace grid or basis functions by neural networks :

$$V_t(x) \approx V_{\theta_t}(x)$$

or directly

$$\pi_t(x) \approx \pi_{\theta_t}(x).$$

Key Idea

Neural networks approximate high-dimensional value functions without discretizing the state space.

Direct Policy Approximation : Han, E, Jentzen (17)

Parametrize policy :

$$\alpha_t = \pi_\theta(t, X_t)$$

Optimize directly :

$$\max_{\theta} \mathbb{E} \left[\sum_{t=0}^{T-1} f(X_t, \pi_\theta(t, X_t)) + g(X_T) \right].$$

Global learning

All time steps optimized jointly.

- Large parameter space
- Memory grows with T
- May converge to poor local minima

Algo NNContP (Bachouch, Huré, Langrené, P. (18)) : control learning by policy iteration

A combination of DP and Han, E, Jentzen algo :

- For $t = T - 1, \dots, 0$: keep track of the approximated optimal policies $\hat{\pi}_s$, $s = t + 1, \dots, T - 1$, and compute

$$\hat{\pi}_t \in \arg \max_{\pi} \mathbb{E} \left[f(X_t, \pi(X_t)) + \sum_{s=t+1}^{T-1} f(\hat{X}_s, \hat{\pi}_s(\hat{X}_s)) + g(\hat{X}_T) \right]$$

where $X_t \sim \mu$ (probability distribution on $\mathcal{X}$), $(\hat{X}_s)_{s=t+1, \dots, T}$, generated from X_t , with policy $(\pi, \hat{\pi}_s)_{s=t+1, \dots, T-1}$. → Practical implementation :

- DNN for policy at time t : $\pi(x) = p(x; \theta) \rightarrow$ optimization over parameter θ
- SGD based on training samples $X_t^{(m)}, (\hat{X}_s^{(m)})_s$, $m = 1, \dots, M \rightarrow \hat{\pi}_t^M = p(\cdot; \hat{\theta}_t^M)$.

Algo NNContP (Bachouch, Huré, Langrené, P. (18)) : control learning by policy iteration

A combination of DP and Han, E, Jentzen algo :

- For $t = T - 1, \dots, 0$: keep track of the approximated optimal policies $\hat{\pi}_s$, $s = t + 1, \dots, T - 1$, and compute

$$\hat{\pi}_t \in \arg \max_{\pi} \mathbb{E} \left[f(X_t, \pi(X_t)) + \sum_{s=t+1}^{T-1} f(\hat{X}_s, \hat{\pi}_s(\hat{X}_s)) + g(\hat{X}_T) \right]$$

where $X_t \sim \mu$ (probability distribution on $\mathcal{X}$), $(\hat{X}_s)_{s=t+1, \dots, T}$, generated from X_t , with policy $(\pi, \hat{\pi}_s)_{s=t+1, \dots, T-1}$. $\rightarrow$ Practical implementation :

- DNN for policy at time t : $\pi(x) = p(x; \theta) \rightarrow$ optimization over parameter θ
- SGD based on training samples $X_t^{(m)}, (\hat{X}_s^{(m)})_s, m = 1, \dots, M \rightarrow \hat{\pi}_t^M = p(\cdot; \hat{\theta}_t^M)$.

Remarks.

1. **No value function iteration** : $\hat{V}_t^M$ is simply computed as the gain functional associated to controls $(\hat{\pi}_s^M)_{s=t, \dots, T-1}$.
2. Low bias estimate, but possibly high variance estimate and large complexity, especially when T is large. See related CV theorem

Actor-Critic

Actor :

$$\pi_{\theta}$$

Critic :

$$V_{\phi}$$

- Actor optimizes policy
- Critic approximates value function
- Monte-Carlo regression for value update

Algo Actor/Critic (Bachouch, Huré, Langrené, P. (18)) (Model-based)

- Initialization : $\hat{V}_T = g$
- For $t = T - 1, \dots, 0$:

(i) Compute the approximated optimal policy

$$\hat{\pi}_t \in \arg \max_{\pi} \mathbb{E} [f(X_t, \pi(X_t)) + \hat{V}_{t+1}(X_{t+1})]$$

where $X_t \sim \mu$, $X_{t+1} \sim P_t(X_t, \pi(X_t))$. Implemented by

- DNN for policy : $\pi(x) = p(x; \theta) \rightarrow$ optimization over parameter θ
- SGD method based on training samples $X_t^{(m)}$, $m = 1, \dots, M \rightarrow \hat{\pi}_t^M = p(\cdot; \hat{\theta}_t^M)$.

Algo Actor/Critic (Bachouch, Huré, Langrené, P. (18)) (Model-based)

- Initialization : $\hat{V}_T = g$
- For $t = T - 1, \dots, 0$:

(i) Compute the approximated optimal policy

$$\hat{\pi}_t \in \arg \max_{\pi} \mathbb{E} [f(X_t, \pi(X_t)) + \hat{V}_{t+1}(X_{t+1})]$$

where $X_t \sim \mu$, $X_{t+1} \sim P_t(X_t, \pi(X_t))$. Implemented by

- DNN for policy : $\pi(x) = p(x; \theta) \rightarrow$ optimization over parameter θ
- SGD method based on training samples $X_t^{(m)}$, $m = 1, \dots, M \rightarrow \hat{\pi}_t^M = p(\cdot; \hat{\theta}_t^M)$.

(ii) Updating : compute the approximated value function from the Bellman equation :

$$\begin{aligned} \hat{V}_t(x) &= \mathbb{E} [f(X_t, \hat{\pi}_t(X_t)) + \hat{V}_{t+1}(X_{t+1}) | X_t = x] \\ &= f(x, \hat{\pi}_t(x)) + \mathbb{E}_{x' \sim P_t(x, \hat{\pi}_t(x))} [\hat{V}_{t+1}(x')] \end{aligned}$$

by Monte-Carlo regression.

Monte-Carlo Regression

Regress now on a set $\mathcal{F}$ of functions on $\mathcal{X}$ (from $t + 1$ to t)

$$\hat{V}_t \in \arg \min_{\Phi \in \mathcal{F}} \mathbb{E} \left| f(X_t, \hat{\pi}_t(X_t)) + \hat{V}_{t+1}(X_{t+1}) - \Phi(X_t) \right|^2$$

- For instance, $\mathcal{F}$ class of DNN : $x \mapsto \Psi(x; \beta)$
- Optimization over θ by SGD based on training samples $X_t^{(m)} \sim \mu, m = 1, \dots, M$,
 $\rightarrow \hat{V}_t^M = \Psi(\cdot; \hat{\beta}_t^M)$.

Convergence of the algo NNcontPI

- M number of training samples
- Neural Network for policy :
 - $\mathcal{A}_K^\gamma$: class of NN (valued in A) with one hidden layer, K **neurons**, and **total variation norm** smaller than γ

Theorem

Under suitable conditions, and assuming the existence of an optimal policy $(\pi_t^*)_t$, we have for all $t = 0, \dots, T - 1$:^a

$$\mathbb{E}_M |V_t(X_t) - \hat{V}_t^M(X_t)| = \underbrace{\mathcal{O}_{\mathbb{P}}\left(\underbrace{\gamma^{T-t} \sqrt{\frac{\ln M}{M}}}_{\text{statistical error}} + \underbrace{\sup_{s=t, \dots, T-1} \inf_{p \in \mathcal{A}_K^\gamma} \|p(X_s) - \pi_s^*(X_s)\|_{L^1}}_{\varepsilon_t^{NN}(\pi^*) : \text{approximation error}}\right)},$$

where $\mathbb{E}_M$ stands for the expectation conditioned on the training set used for computing the approximated optimal policies $\hat{\pi}_s^M$, and $(X_s)_s$ is the corresponding controlled process starting from $X_t \sim \mu$.

^a. $x_M = \mathcal{O}_{\mathbb{P}}(y_M)$ means that $\exists c > 0$ s.t. $\mathbb{P}[|x_M| \geq c|y_M|] \rightarrow 0$ as M goes to infinity

Convergence of the algo Actor/Critic

- M number of training samples
- Neural Network for policy and value function :
 - $\mathcal{A}_K^\gamma$: class of NN (valued in A) with one hidden layer, K **neurons**, and **total variation norm** smaller than γ
 - $\mathcal{F}_K^\gamma$: class of NN (valued in $\mathbb{R}$) with one hidden layer, K **neurons**, and **total variation norm** smaller than γ

Theorem

Under suitable conditions, and assuming the existence of an optimal policy $(\pi_t^*)_t$, we have for all $t = 0, \dots, T-1$:

$$\mathbb{E}_M |V_t(X_t) - \hat{V}_t^M(X_t)| = \mathcal{O}_{\mathbb{P}} \left(\left(\gamma^4 K \frac{\ln M}{M} \right)^{\frac{1}{2(T-t)}} + \underbrace{\sup_{s=t, \dots, T-1} \inf_{p \in \mathcal{A}_K^\gamma} \|p(X_s) - \pi_s^*(X_s)\|_{L^1}}_{\varepsilon_t^{NN}(\pi^*)} + \underbrace{\sup_{s=t, \dots, T} \inf_{\Psi \in \mathcal{F}_K^\gamma} \|\Psi(X_s) - V_s(X_s)\|_{L^2}}_{\varepsilon_t^{NN}(V)} \right)^{\frac{1}{2(T-t)}}.$$

Model setup

Real-options valuation of gas storage :

- Gas (random) price $(P_t)_t$
- Gas inventory $(C_t)_t$ controlled by the decision α_t to inject, do nothing, or withdraw gas :

$$C_{t+1} = \begin{cases} C_t + b_{in} & \text{if } \alpha_t = +1 & \text{(injection/buy gas)} \\ C_t & \text{if } \alpha_t = 0 & \text{(do nothing)} \\ C_t - s_{out} & \text{if } \alpha_t = -1 & \text{(withdraw/sell gas)} \end{cases}$$

with $b_{in}, s_{out} > 0$.

- Physical inventory constraint :

$$C_t \in [C_{min}, C_{max}].$$

Control problem

- Maximize over α on finite horizon T :

$$\mathbb{E} \left[\sum_{t=0}^{T-1} f(P_t, C_t, \alpha_t) + g(P_T, C_T) \right]$$

with

- Revenue at any time t :

$$f(p, c, a) = \begin{cases} -b_{in}p - \kappa c & \text{if } a = +1 & \text{(injection/buy gas)} \\ -\kappa c & \text{if } a = 0 & \text{(do nothing)} \\ s_{out}p - \kappa c & \text{if } a = -1 & \text{(withdraw/sell gas)} \end{cases}$$

with storage cost $\kappa > 0$.

- Terminal condition : penalization for having less gas than initially

$$g(p, c) = -\mu p(C_0 - c)_+.$$

with $\mu > 0$.

Numerical results

- **Model parameters :**

- Mean-reverting gas price around $\bar{p} = 5$, with rate $\beta = 0.5$

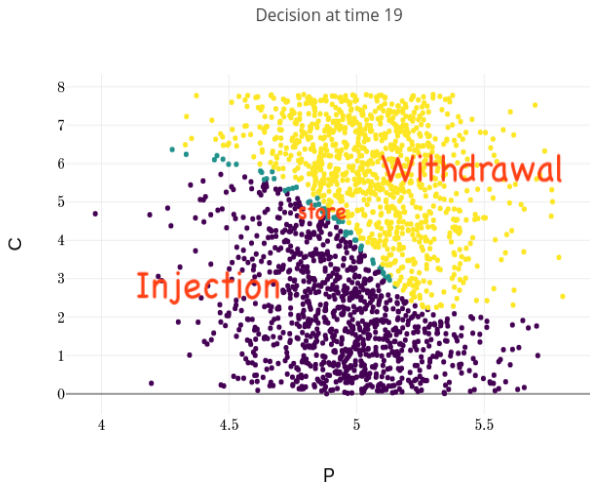
$$P_{t+1} = \bar{p}(1 - \beta) + \beta P_t + \xi_{t+1}, \quad \xi_t \sim \mathcal{N}(0, \sigma^2 = 0.05), \quad P_0 = 4,$$

- $b_{in} = 0.06$, $s_{out} = 0.25$, $\kappa = 0.01$,
- $T = 30$, $\mu = 2$, $C_0 = 4$, $C_{min} = 0$, $C_{max} = 8$.

- Implementation by [Algo NNContPI with DNN classification](#) :

- 3 hidden layers with $15 + 15 + 5$ neurons, output layer with 3 neurons
- ELU activation function
- Training samples of size $M = 250000$

Optimal policy regions



ClassifPI-estimated optimal policy regions varying in time



Concluding remarks

- Deep learning methods provide a breakthrough for the computation challenge of solving high dimensional nonlinear problem arising in PDEs and stochastic control
- Extensions concern :
 - PDEs in non Markovian systems (Path-dependent PDEs) : rough volatility and Volterra equations
 - Stochastic control in large population model with interacting agents : mean-field game, mean-field control, with natural applications in finance.

From Model-Based to Reinforcement Learning

Lecture 2 :

- Known model
- Dynamic programming
- Deterministic policies

Lecture 3 :

- Unknown model
- Learning from data
- Randomized policies

References

- ① A. Bachouch, C. Huré, N. Langrené, H. Pham : Deep neural networks algorithms for stochastic control problems on finite horizon, part II, numerical applications. *Methodology and Computing in Applied Probability*, 2022, vol. 24, pp. 143-178.
- ② J. Han, A. Jentzen, W. E : Solving high-dimensional partial differential equations using deep learning, *Proc. National Acad. Sci.*, 2017.
- ③ C. Huré, H. P, A. Bachouch, N. Langrené : Deep neural networks algorithms for stochastic control problems on finite horizon, part I : convergence analysis, *SIAM J. Numerical Analysis*, 2021, vol. 59, pp. 525-557.
- ④ C. Huré, H. Pham, X. Warin : Deep backward schemes for high-dimensional nonlinear PDEs, *Mathematics of Computation*, 2020, 89(324), 1547-1580
- ⑤ M. Germain, H. Pham, X. Warin : Neural networks-based algorithms for stochastic control and PDEs in finance, *Machine Learning and Data Science for Financial Markets : a guide to contemporary practices*, 2023, Cambridge University Press, Editors : A. Capponi and C-A. Lehalle.
- ⑥ M. Sabate Vidales, D. Siska, L. Szpruch : Martingale functional control variates via deep learning, arXiv :1810.05094v1
- ⑦ J. Sirignano, K. Spiliopoulos : DGM : a deep learning algorithm for solving PDEs, *J. Comput. Physics*, 2017, 375.

Annex : Bellman equations

[back](#)

- For a fixed policy $\pi = (\pi_t)_t$,

$$\begin{cases} V_T^\pi(x) = g(x) \\ V_t^\pi(x) = \mathbb{E}_{a \sim \pi_t(x)} \left[\underbrace{f(x, a) + \mathbb{E}_{x' \sim P_t(x, a)} [V_{t+1}^\pi(x')]}_{= Q_t^\pi(x, a)} \right], \quad t = 0, \dots, T-1, \end{cases} \quad (9)$$

and

$$\begin{cases} Q_T^\pi(x, a) = g(x) \\ Q_t^\pi(x, a) = f(x, a) + \mathbb{E}_{\substack{x' \sim P_t(x, a) \\ a' \sim \pi_{t+1}(x')}} [Q_{t+1}^\pi(x', a')] \end{cases} \quad (10)$$

- For optimal value and Q functions

$$\begin{cases} V_T(x) = g(x) \\ V_t(x) = \sup_{a \in A} \left\{ \underbrace{f(x, a) + \mathbb{E}_{x' \sim P_t(x, a)} [V_{t+1}(x')]}_{= Q_t(x, a)} \right\} \end{cases} \quad (11)$$

and

$$\begin{cases} Q_T(x, a) = g(x) \\ Q_t(x, a) = f(x, a) + \mathbb{E}_{x' \sim P_t(x, a)} \left[\sup_{a' \in A} Q_{t+1}(x', a') \right] \end{cases} \quad (12)$$

→ Optimal (deterministic) policy : $\pi^* = (\pi_t^*)_t$ from the Q -value function :

$$\pi_t^*(x) \in \arg \max_{a \in A} Q_t(x, a).$$

- From definition of Q^π , and law of conditional expectation, we get :

$$\begin{aligned}
 Q_t^\pi(x, a) &= \mathbb{E}_\pi \left[f(X_t, \alpha_t) + \underbrace{\mathbb{E}_\pi \left[\sum_{s=t+1}^{T-1} f(X_s, \alpha_s) + g(X_T) \mid X_{t+1} \right]}_{V_{t+1}^\pi(X_{t+1})} \mid X_t = x, \alpha_t = a \right] \\
 &= f(x, a) + \mathbb{E}_{x' \sim P_t(x, a)} [V_{t+1}^\pi(x')].
 \end{aligned} \tag{13}$$

By using the relation $V_s^\pi(x) = \mathbb{E}_{a \sim \pi_s(x)} [Q_s^\pi(x, a)]$ for $s = t$, this yields the Bellman equation (9) for V^π , and for $s = t + 1$, this gives the Bellman equation (10) for Q^π .

- For the Q -value function, and using (13), we have

$$\begin{aligned}
 Q_t(x, a) &= \sup_\pi Q_t^\pi(x, a) = \sup_\pi \left\{ f(x, a) + \mathbb{E}_{x' \sim P_t(x, a)} [V_{t+1}^\pi(x')] \right\} \\
 &= f(x, a) + \mathbb{E}_{x' \sim P_t(x, a)} [V_{t+1}(x')],
 \end{aligned} \tag{14}$$

by definition of $V = \sup_\pi V^\pi$.

Annex : Proof of Bellman (Ctd)

[back](#)

- For the optimal value function, and using (9), we have

$$\begin{aligned}
 V_t(x) &= \sup_{\pi} V_t^{\pi}(x) = \sup_{\pi} \mathbb{E}_{a \sim \pi_t(x)} \left[f(x, a) + \mathbb{E}_{x' \sim P_t(x, a)} [V_{t+1}^{\pi}(x')] \right] \\
 &\leq \sup_{a \in A} \left[f(x, a) + \mathbb{E}_{x' \sim P_t(x, a)} [V_{t+1}(x')] \right] = \sup_{a \in A} Q_t(x, a), \quad (15)
 \end{aligned}$$

by (14). To prove the reverse inequality, let us consider the (deterministic) policy π^* defined by $\pi_t^*(x) \in \arg \max_a Q_t(x, a)$ for all t, x . Then, we prove by backward induction that for all $t = 0, \dots, T$,

$$V_t^{\pi^*}(x) \geq \sup_{a \in A} Q_t(x, a) \geq V_t(x), \quad x \in \mathcal{X}.$$

For $t = T$, this is clear since $V_T^{\pi^*} = Q_T = V_T = g$. Suppose now that $V_{t+1}^{\pi^*} \geq \sup_{a \in A} Q_{t+1}(\cdot, a) \geq V_{t+1}$. From the Bellman equation (9) for V^{π} , we then have

$$\begin{aligned}
 V_t^{\pi^*}(x) &= f(x, \pi_t^*(x)) + \mathbb{E}_{x' \sim P_t(x, \pi_t^*(x))} [V_{t+1}^{\pi^*}(x')] \\
 &\geq f(x, \pi_t^*(x)) + \mathbb{E}_{x' \sim P_t(x, \pi_t^*(x))} [V_{t+1}(x')] = Q_t(x, \pi_t^*(x)) = \sup_{a \in A} Q_t(x, a) \geq V_t(x),
 \end{aligned}$$

by (14), (15), hence the required relation at t . Since $V^{\pi^*} \leq V$, this shows the equality $V_t(x) = \sup_{a \in A} Q_t(x, a) = V_t^{\pi^*}(x)$ for all t, x , hence the optimality of π^* . Together with (14), this yields the Bellman equation (11) for V , and also the Bellman equation (12) for Q .

Let $Y \in L^2(\Omega, \mathbb{P})$, and X some random variable on $(\Omega, \mathbb{P})$ valued in $\mathbb{R}^d$. Then,

$$\mathbb{E}[Y|X] = \hat{\varphi}(X),$$

where $\hat{\varphi}$ is a measurable function on $\mathbb{R}^d$ that minimizes the L^2 -error

$$\mathbb{E}|Y - \varphi(X)|^2,$$

over measurable functions φ on $\mathbb{R}^d$, s.t. $\mathbb{E}|\varphi(X)|^2 < \infty$.